

Patient Portal — Mental Health Clinic

Project plan for Nakisa

Version 1.2 · September 2026

1. Project objective

Meridian Psychological Services is a mid-size outpatient clinic with 11 clinicians and roughly 400 active patients. Many patients have difficulty remembering what was discussed in therapy between sessions, while the clinic's existing progress notes are written primarily for clinical documentation rather than patient understanding.

The proposed portal gives each patient secure access to a plain-language summary of eligible session material. The summary can also connect the current session with themes or concepts that appeared in earlier sessions.

A typical patient experience would look like this:

This session focused on the physical symptoms you notice before presentations at work, including chest tightness and racing thoughts. You and your clinician practiced a grounding technique to use in the moment. You also revisited "catastrophizing," a pattern discussed in earlier sessions where you jump to the worst-case outcome. The plan was to try the grounding technique twice before your next presentation and note how it went.

The historical connection is the main feature that makes this more useful than a one-off note summarizer.

MVP goal

Build a working prototype, using synthetic data only, that demonstrates:

- secure patient login and data isolation;
- upload and processing of approved document types;
- plain-language session summaries;
- links to relevant concepts from prior sessions;
- clinician review before a summary is released; and

The prototype should be suitable for a clinic demo, but it should not be represented as production-ready for real patient data.

2. Scope

In scope

- Patient-facing web portal for login, upload, session history, and released summaries
- Authentication tied to an opaque patient identifier
- Per-patient storage and retrieval isolation
- Document ingestion and eligibility checks
- Plain-language summarization and concept extraction
- Retrieval of relevant prior patient history
- Minimal clinician/admin review queue for approve, edit, and reject
- Audit records for access and release actions
- Storage and compute on GCP

Out of scope for the MVP

- Full clinician portal
- EHR integration
- Patient-clinician messaging
- Scheduling and billing
- Mobile applications
- Diagnosis, prognosis, or treatment recommendations

- Open-ended chatbot functionality
- Use of real PHI before compliance and security requirements are completed

3. Key design constraints

These constraints should be agreed before implementation because they affect the data model, workflow, and architecture.

3.1 Document eligibility

Mental health records can include document types with different disclosure and access rules. The system therefore needs an explicit `note_type` or `document_type` field from the start.

For the prototype:

- every uploaded document receives a document-type label;
- processing is allowed only for types on a clinic-approved list;
- an optional classifier can flag a likely mismatch, but the model should not be the authority that decides whether a document is eligible for patient release; and
- ineligible documents stop before summarization.

Before real patient records are used, the clinic's compliance or legal team should confirm which document types can be processed and released. This plan is a technical design, not legal advice.

3.2 Synthetic data first

The initial build should use synthetic session notes only. This keeps the engineering work separate from the additional controls required for real PHI and allows the full workflow to be demonstrated without production access.

A production deployment would require a separate compliance and security readiness review, including confirmation of the clinic's cloud agreements, service eligibility, audit requirements, encryption controls, retention policy, incident response process, and access model.

3.3 Human review before release

Generated summaries should not be shown to patients automatically in the MVP.

Session notes may contain sensitive or easily misrepresented material, including risk content, diagnostic language, third-party information, or clinically hedged statements. The system should therefore create each summary with status: `pending_review`.

A minimal review screen is enough for the MVP. It only needs:

- a queue of pending summaries;
- the source text or relevant source excerpt;
- the generated summary and flags;
- approve, edit, and reject actions; and
- a record of who reviewed and released the summary.

This is intentionally smaller than a full clinician portal.

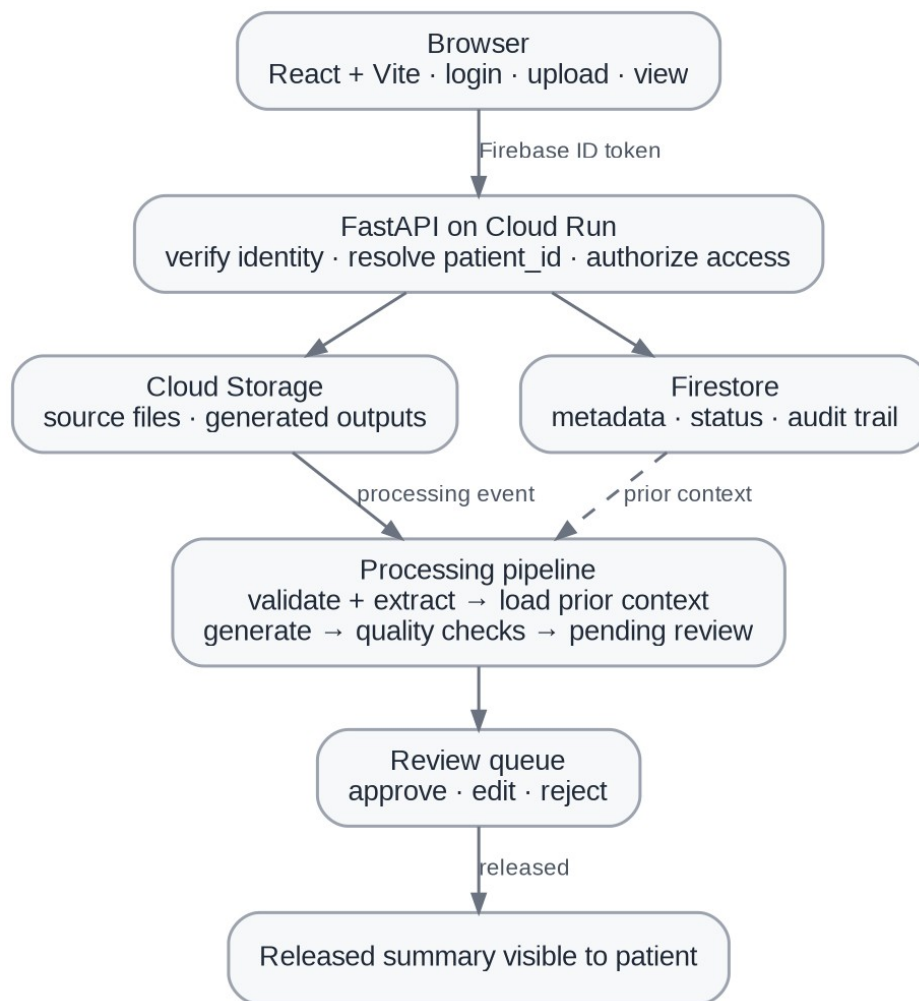
3.4 Patient isolation

Cross-patient exposure is the highest-severity technical failure in this system. Patient identity must be derived from the authenticated session on the server, never accepted from client-supplied parameters.

The same patient boundary must apply to:

- Firestore queries;
- Cloud Storage paths;
- historical retrieval;
- generated summaries;
- signed URLs; and
- audit records.

4. Proposed architecture



Architecture choices

Cloud Run rather than an always-on VM for the API. The prototype has low and intermittent traffic, so serverless compute is a better fit and avoids paying for an idle application server.

No patient IAM access to the storage bucket. The browser may upload through a short-lived signed URL, but only after the API authorizes the request and scopes the URL to one object. Patients do not receive general bucket permissions or choose their own storage paths.

Model access behind an interface. The summarization code should not depend directly on one provider. A thin backend interface makes it possible to start with Vertex AI and switch to a self-hosted endpoint later without rewriting the pipeline.

5. Patient application

5.1 Front end

Suggested stack: React, Vite, TypeScript, Tailwind. Deploy with Firebase Hosting or Cloud Run.

The MVP needs four patient-facing screens:

Screen	Main contents
Login	Email/password or approved passwordless flow
Dashboard	Reverse-chronological session list with date, status, and preview
Session detail	Released summary, recurring concepts, and links to related prior sessions

Screen	Main contents
Upload	File selection, document type, validation, progress, and final status

The interface should be simple and low-friction. Avoid dense tables, technical processing labels, and unnecessary alerts. Status wording should be understandable to patients, for example:

- `pending_review` → “Being reviewed”
- `released` → “Ready to read”
- `failed` → “Could not be prepared”

Each released summary should include a persistent notice such as:

This summary was generated with AI and reviewed by your care team. It is intended to help you remember your session and does not replace discussion with your clinician.

Patients should also have a visible way to flag a summary as inaccurate or confusing.

5.2 API

Suggested stack: FastAPI on Cloud Run, with Firestore for metadata.

Core endpoints:

```
GET /api/sessions
GET /api/sessions/{id}
POST /api/uploads/initiate
POST /api/uploads/finalize
POST /api/sessions/{id}/flag

GET /admin/queue
POST /admin/summaries/{id}/release
POST /admin/summaries/{id}/reject
```

If cookie-based sessions are preferred, an authentication exchange endpoint can be added. Otherwise, the API can verify Firebase ID tokens directly.

5.3 API authorization rules

The API should follow three rules consistently:

1. `patient_id` is derived from the verified identity token on the server.
2. Client-supplied patient identifiers are ignored for authorization.
3. Every query or object path that contains patient data is built from the server-derived identifier.

For patient-facing resources, an unauthorized request for another patient's document should return a not-found response rather than confirming that the resource exists.

6. Authentication, storage, and isolation

6.1 Identity

Use Firebase Authentication or Google Cloud Identity Platform rather than implementing password management in the application.

For the MVP, clinic-controlled enrollment is preferable to open self-service registration. Each authenticated user is associated with an opaque UUID-style `patient_id`; names, dates of birth, initials, or medical record numbers should not be used in storage paths.

The mapping can be stored as a custom claim or in a server-side identity mapping table. If custom claims are used, the API verifies them on every request.

6.2 Storage layout

A simple storage layout is sufficient:

```
gs://meridian-patient-data/  
patients/  
  {patient_id}/  
    raw/{document_id}.{ext}  
    summaries/{document_id}.json
```

Firestore stores document metadata, processing status, review status, timestamps, and audit events.

6.3 Upload flow

Support .txt, .md, .docx, and .pdf, with a prototype limit of 10 MB.

Recommended flow:

1. Patient selects a file and document type.
2. Browser requests an upload from the API.
3. API authenticates the user and creates the exact patient-scoped object path.
4. API returns a short-lived signed URL for that object only.
5. Browser uploads directly to Cloud Storage.
6. Browser calls the finalize endpoint.
7. API validates that the object exists and matches the expected type and size.
8. API creates the Firestore record and enqueues processing.

The browser never chooses the patient storage prefix itself.

6.4 Isolation tests

Isolation should have its own automated test suite and be treated as a release gate.

At minimum, test:

- Patient A requesting Patient B's document ID
- patient ID injected through body, query string, or headers
- expired token
- revoked token
- missing or invalid patient mapping
- expired signed URL
- signed URL used for the wrong object
- path traversal attempts in document IDs
- historical retrieval returning only the authenticated patient's records

7. Summarization and historical context

7.1 Model choice for the prototype

Use a hosted model through Vertex AI for the first version. It reduces operational work and is a better match for low-volume prototyping than maintaining a dedicated model server.

Keep the model call behind a small interface:

```
class SummarizerBackend(Protocol):  
    def summarize(  
        self,  
        note: str,  
        context: list[PriorSummary],  
    ) -> SummaryResult:  
        ...
```

A self-hosted model can be revisited if the clinic later has a specific data residency, cost, customization, or model-control requirement.

7.2 Processing pipeline

Step 1 — Validate and extract

- Confirm the uploaded document has an approved document type.
- Extract text using the appropriate parser.
- If the declared type and content appear inconsistent, flag the document for review rather than letting the model override the clinic-supplied type.

Step 2 — Load patient history

For the MVP, do not introduce a vector database unless it is needed.

A patient with a modest history can be handled by loading:

- the current note;
- the most recent prior summaries; and
- a compact running list of previously identified concepts.

This is easier to audit than semantic retrieval and makes it clear what context was available to the model.

If embeddings are introduced later, `patient_id` must be an explicit retrieval boundary. Separate per-patient namespaces are preferable to relying only on similarity or loosely applied metadata filters.

Step 3 — Generate structured output

Return structured JSON rather than free-form text:

```
{
  "summary": "Plain-language narrative, 150-250 words",
  "concepts": [
    {
      "term": "catastrophizing",
      "plain_language": "Jumping to the worst-case outcome",
      "first_seen": "2026-02-14",
      "recurring": true
    }
  ],
  "connections_to_history": [
    {
      "prior_session": "2026-04-03",
      "relationship": "Same pattern discussed"
    }
  ],
  "flags": ["contains_risk_content"]
}
```

Generation rules should be explicit and testable:

- use plain language and second person;
- target roughly grade 6–8 readability;
- explain necessary clinical terms;
- do not add a diagnosis, prognosis, or treatment recommendation;
- do not infer facts that are not supported by the source material;
- do not name third parties unless there is a specific approved reason to do so; and
- return a limited or empty result when the source is too sparse rather than filling gaps.

For every historical connection, store the prior session ID that supported it. This gives the reviewer traceability and makes the feature easier to debug.

Step 4 — Quality and safety checks

Run automated checks before the item reaches the review queue:

- unsupported-claim or faithfulness check;
- risk-content flagging;
- diagnostic-language flagging;
- third-party name detection;
- readability score; and
- schema validation.

These checks should route or flag content; they should not replace clinician review.

Step 5 — Persist pending output

Store the generated result with:

```
status = pending_review
```

It is not available through patient-facing endpoints until released.

Step 6 — Review and release

The reviewer can approve, edit, or reject the summary. On approval:

```
status = released
```

The released version, reviewer identity, timestamp, and any edits should be retained for audit and troubleshooting.

7.3 Processing jobs

Use Cloud Run Jobs or a Cloud Run worker triggered from Pub/Sub after upload finalization.

The job should support retry with backoff and a dead-letter path after repeated failure. Failed items should have a clear status in the patient UI rather than remaining indefinitely in a processing state.

8. Delivery plan

Phase	Deliverable
0. Foundation	GCP project, repo, Firestore, bucket, service accounts, CI, initial synthetic notes
1. Auth + isolation	Authentication, patient mapping, authorization dependency, isolation tests
2. Upload + storage	Signed upload flow, validation, metadata records, upload UI
3. Base summarization	Extract, validate type, summarize, store pending output
4. Historical context	Prior-summary context, recurring concepts, session links, provenance
5. Safety + review gate	Automated flags, review queue, approve/edit/reject, audit records
6. Patient experience	Dashboard, session detail, status states, flag mechanism, polish

Suggested MVP acceptance criteria

The MVP is ready to demo when:

- all isolation tests pass;
- no patient-facing endpoint returns another patient's data;
- only approved document types enter the summarization path;
- generated summaries remain pending until reviewed;
- historical links point to the correct patient and prior session in the test set;
- failed processing jobs surface a visible status; and
- the complete Dana demo flow can be run without manual database edits.

9. Cost assumptions

Prototype

With synthetic data and low usage, infrastructure cost should remain small because Cloud Run can scale down when idle and the storage footprint is limited. Model calls are likely to be the most visible variable cost during development.

Production

A production deployment should be estimated separately. Infrastructure may still be modest at this clinic size, but engineering, compliance, security review, monitoring, and operational support will be the larger cost drivers.

Do not quote production numbers from this plan without rechecking current GCP and model pricing.

10. Demo flow

The demo should focus on the two things the system needs to prove: useful longitudinal context and safe patient isolation.

1. Log in as Dana and show several months of prior sessions.
2. Open an older session that includes the concept “catastrophizing.”
3. Upload a new synthetic note where the same concept appears again.
4. Show the new item move to `pending_review`.
5. Open the review queue, inspect the generated summary and historical connection, make a small edit, and release it.
6. Return to Dana’s dashboard and show the released summary with its link to the earlier session.
7. Attempt to request another patient’s session ID and show that the application returns no data.

This demonstrates the product value and the core security boundary without relying on production data.